PlaneSpotter Social App

Week 1: React foundations and a mock sightings feed

Your goal	Create a React app that runs locally and renders reusable aircraft sighting cards from hardcoded mock data. Do not add Django, a database, authentication or API calls this week.

Time budget

Session	Focus	Time
1	Project setup and first run	1 hour
2	Components: Navbar and SightingCard	1 hour
3	Props and reusable cards	1 hour
4	Mock data and array mapping	1 hour
5	Basic CSS, review and Git checkpoint	1 hour

Definition of done

[ ] The Vite React app runs at localhost:5173.

[ ] The page includes a Navbar and at least three SightingCard components.

[ ] Each card receives its data through props.

[ ] Cards are generated from a mock sightings array using map().

[ ] The layout has basic readable styling.

[ ] The Week 1 work is committed to Git.

Session 1 - Set up React

1. Open Terminal and check that Node.js and npm are available:

```
node --version
npm --version
```

If Node.js is missing on your Mac and Homebrew is available, install it with:

```
brew install node
```

2. Create the project. From the folder where you keep development projects, run:

```
npm create vite@latest planespotter-frontend -- --template react
cd planespotter-frontend
npm install
npm run dev
```

3. Open the local address shown by Vite, normally:

```
http://localhost:5173
```

Checkpoint	You should see the default Vite React page. Keep the terminal running while you work.

4. Open the project in your editor. If you use Visual Studio Code:

```
code .
```

5. Simplify **src/App.jsx** so you have a clean starting point:

```
function App() {
return (
<main>
<h1>PlaneSpotter NZ</h1>
<p>Recent aircraft sightings</p>
</main>
);
}

export default App;
```

Session 1 checklist

[ ] The app starts without errors.

[ ] You can explain that Vite is the development/build tool and React creates the UI.

[ ] Changing App.jsx changes what appears in the browser.

Session 2 - Build components

Create a components folder:

```
src/components/
```

Create **src/components/Navbar.jsx**:

```
function Navbar() {
return (
<nav className="navbar">
<h1>PlaneSpotter NZ</h1>
</nav>
);
}

export default Navbar;
```

Create **src/components/SightingCard.jsx**:

```
function SightingCard() {
return (
<article className="sighting-card">
<div className="photo-placeholder">Aircraft photo</div>
<h2>ZK-NZE</h2>
<p>Boeing 787-9</p>
<p>Wellington Airport</p>
</article>
);
}

export default SightingCard;
```

Import and render both components in **src/App.jsx**:

```
import Navbar from "./components/Navbar";
import SightingCard from "./components/SightingCard";

function App() {
return (
<>
<Navbar />
<main className="feed">
<SightingCard />
</main>
</>
);
}

export default App;
```

Concept	A component is a reusable piece of UI. In JSX, your custom component is used like an HTML element: <SightingCard />.

Session 3 - Make the card reusable with props

Replace the hardcoded card values with props. Update **SightingCard.jsx**:

```
function SightingCard({ registration, aircraftType, airport, spotter }) {
return (
<article className="sighting-card">
<div className="photo-placeholder">Aircraft photo</div>
<h2>{registration}</h2>
<p>{aircraftType}</p>
<p>{airport}</p>
<small>Spotted by {spotter}</small>
</article>
);
}

export default SightingCard;
```

Pass those values from **App.jsx**:

```
<SightingCard
registration="ZK-NZE"
aircraftType="Boeing 787-9"
airport="Wellington Airport"
spotter="Hyacinthe"
/>
```

Concept	Props are data passed from a parent component into a child component. The card controls the layout; the parent supplies the aircraft sighting data.

Experiment

Render a second card with different values. Confirm that you reused the same component without copying its internal markup.

Session 3 checklist

[ ] The card displays different values when its props change.

[ ] You understand that props flow into the component.

[ ] You did not create a separate component for each aircraft.

Session 4 - Render mock data with map()

Add this array near the top of **App.jsx**:

```
const sightings = [
{
id: 1,
registration: "ZK-NZE",
aircraftType: "Boeing 787-9",
airport: "Wellington Airport",
spotter: "Hyacinthe"
},
{
id: 2,
registration: "ZK-OKQ",
aircraftType: "Airbus A320",
airport: "Auckland Airport",
spotter: "Alex"
},
{
id: 3,
registration: "ZK-MVA",
aircraftType: "ATR 72-600",
airport: "Christchurch Airport",
spotter: "Jamie"
}
];
```

Replace the single card in the feed with:

```
<section className="card-grid">
{sightings.map((sighting) => (
<SightingCard
key={sighting.id}
registration={sighting.registration}
aircraftType={sighting.aircraftType}
airport={sighting.airport}
spotter={sighting.spotter}
/>
))}
</section>
```

Concept	map() creates one SightingCard for each object in the sightings array. The key helps React identify each item consistently.

Session 4 checklist

[ ] Three cards display from one array.

[ ] Adding another object to the array creates another card.

[ ] Every sighting has a unique id used as the React key.

Session 5 - Add simple styling

Replace or simplify **src/index.css** with:

```
* {
box-sizing: border-box;
}

body {
margin: 0;
font-family: Arial, sans-serif;
background: #f4f7fa;
color: #172033;
}

.navbar {
background: #17365d;
color: white;
padding: 1rem 2rem;
}

.feed {
max-width: 1100px;
margin: 0 auto;
padding: 2rem;
}

.card-grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
gap: 1rem;
}

.sighting-card {
background: white;
border-radius: 12px;
padding: 1rem;
box-shadow: 0 4px 14px rgba(0, 0, 0, 0.08);
}

.photo-placeholder {
min-height: 140px;
display: grid;
place-items: center;
background: #dbe6ef;
border-radius: 8px;
color: #486581;
}
```

Review your work

[ ] Navbar is clearly separated from the page content.

[ ] Cards are readable on both wide and narrow browser windows.

[ ] No errors appear in the browser console.

[ ] The page still renders after stopping and restarting npm run dev.

Save a Git checkpoint

```
git init
git add .
git commit -m "Week 1 - React sightings feed"
```

If Git asks for your name or email, follow the instructions printed in Terminal, then run the commit again.

End-of-week review

You should now be able to explain these concepts in your own words:

Concept	Your explanation
Component	A reusable piece of user interface.
Props	Data passed into a component.
JSX	HTML-like syntax used to describe React UI.
map()	Transforms each array item into rendered output.
key	A stable identifier React uses for list items.
Vite	The tool running and building the frontend project.

Do not add yet

Leave these for later weeks: Django, PostgreSQL, API requests, login, image uploads, likes, comments, global state libraries, TypeScript and advanced styling frameworks.

Ready for Week 2 when...	You can add a new sighting object, pass data through props, and understand why map() creates another card. Week 2 will introduce state, form events and routing.